SHRUSHTI GADPAYALE

EDUCATION

CDAC-Post Graduate Diploma (PG-DESD) 2021
CDAC ACTS,Pune Grade A

Bachelor Of Engineering (Electrical Engineering), RTMNU University 2015-2019
Datta Meghe Institute of Engineering, Technology Research, Wardha 7.2/10.0

SKILLS

Technical Skills

Embedded C, C++, System Programming, Device Drivers, Linux Kernel Internals, Custom Device Driver Development, Kernel Module Development, procfs/sysfs
UART, I2C, SPI, CAN, SOME/IP, DDS, IPC, AMWSL, USB
Linux, Window and QNX Device Driver, and kernel architecture
Performance Evaluation (SomeIP and IPC) and Functionality Evaluation, AUTOSAR, ASPICE, Functional Cluster Architecture, COM/Signal Modules, Middleware Integration

Tools

Arxml Configuration-C4K tool (compose tool), CMake, Wireshark
DLT tool, Slogger tool, Canoe, CanApe(CanAnalyzer), Debugging tools - QNX Momentics, JTAG, GDB and Valgrind
Hardware Setup and BSP Flashing - Evaluation board and ELITE ECU (RTOS) and STM32, Renesas R-Car Gen4, Qualcomm SoCs.
Jira, Confluence, Enterprise Architect (EA Tool), BSP Flashing (Evaluation Boards, ELITE ECU)

EXPERIENCE

KPIT - Embedded Engineer Sept 2025 - April 2026
Pune

- Configured functional clusters and probe applications leveraging DDS and AMWSL inter-process communication (IPC) protocols for HProbe0 and HProbe1; applied strong OS-level IPC fundamentals (C/C++) within an embedded Linux automotive software stack.
- Architected and maintained SWE1/SWE2 software design artifacts — Class, Sequential, and Interface diagrams — using Enterprise Architect (EA Tool), following ASPICE-compatible development practices to ensure traceability from requirements to implementation.
- Developed and integrated middleware components for automotive-grade embedded Linux platforms in C/C++, enabling reliable inter-functional-group communication and meeting real-time determinism requirements.
- Collaborated cross-functionally with system architects and test teams, translating functional requirements into robust software designs and proactively resolving integration defects at the middleware layer.

KPIT - Embedded Engineer(Onsite) Sept 2024 - Sept 2025
Tokyo, Japan

- Acted as onsite embedded software engineer at Honda, diagnosing and resolving complex runtime defects across middleware, functional clusters, and application integration layers in a Linux/QNX-based IVI and PADAS platform (Renesas R-Car Gen4).
- Designed and implemented multi-threaded C++ components within the Crypto Daemon service, applying mutex-based synchronization, thread lifecycle management, and priority-aware scheduling to ensure data-integrity and deadlock-free operation in a concurrent QNX/RTOS environment.

- Performed hands-on validation and runtime debugging on RTOS-based ECUs using CANoe and CANape, verifying platform stability across both simulated and live hardware environments; reproduced and resolved intricate software defects under tight delivery schedules.
- Leveraged GDB, DLT Tool, and QNX Momentics for in-depth kernel and user-space debugging, isolating root causes in multi-process embedded systems and driving fixes through to production closure.
- Managed sprint cycles, issue tracking, and client communication via Jira and Confluence, maintaining clear delivery visibility across distributed teams.

KPIT - Embedded Engineer Oct 2021 - Oct 2024
Pune, India

- Developed a SOME/IP protocol-based sample application in C++ to validate EXM module functionality, deepening hands-on expertise in middleware IPC, service-oriented communication, and embedded Linux integration.
- Led end-to-end design and implementation of sample applications for KSAR Functional Groups (FGs), validating COM and Signal module behavior on Linux/QNX embedded targets; applied multi-threaded programming patterns with semaphores and mutexes for reliable inter-task communication.
- Authored comprehensive Integration Test Plans (ITP) — including test cases, procedures, and validation strategies for multiple Functional Groups — in line with ASPICE software development practices, ensuring full requirement coverage and audit-readiness.
- Executed end-to-end hardware validation on ECUs and Renesas RCAR evaluation boards: from hardware connections and BSP flashing (Qualcomm SoCs) to runtime verification using JTAG, GDB, and Valgrind, identifying memory leaks and confirming functional correctness.
- Built foundational expertise in Linux kernel internals, device driver architecture (I2C, SPI, UART subsystems), and ARM Cortex-M platforms through driver development projects, directly supporting platform bring-up and debugging activities.

PROJECTS

Process Information driver Designed and implemented a custom procfs device driver in Linux to expose critical process-level information — including PID, TGID, and kernel stack addresses — through the /proc filesystem interface. Leveraged knowledge of Linux kernel architecture and kernel module development to write, compile, and load the driver dynamically. Utilized GDB and Valgrind for debugging and memory validation during development. Ensured compatibility with ARM Cortex-M-based environments and validated driver behavior across multiple process scenarios to confirm accurate data retrieval.

I2C Driver STM-32 Developed a low-level I2C peripheral driver from scratch for the STM32 microcontroller series using Embedded C, enabling reliable communication between the STM32 and I2C-compatible slave devices. Implemented complete bus initialization, master/slave role configuration, and bidirectional data transfer using both interrupt-driven and polling mechanisms. Applied UART-based serial logging for real-time debugging and verification of data transactions. Validated driver functionality using JTAG debugging tools and ensured timing compliance with the I2C protocol specification across various clock speed configurations.

Vehicle Control System Implemented using FREERTOS Developed a basic GUI/dashboard (using Python/web interface) to monitor vehicle parameters such as speed, collision alerts, and system status in real time. Architected and implemented a multi-featured Vehicle Control System on BeagleBone Black using FreeRTOS, demonstrating real-time task scheduling and inter-task communication in an automotive-grade environment. Developed independent RTOS tasks for Forward Collision Detection, Hill Assist, Cruise Control, and Speech Recognition, ensuring deterministic execution through priority-based scheduling and semaphores. Integrated sensor inputs and actuator control logic using SPI and UART communication protocols. Conducted end-to-end system validation by simulating real driving scenarios, verifying task synchronization, and ensuring fault-tolerant behavior across all functional modules.